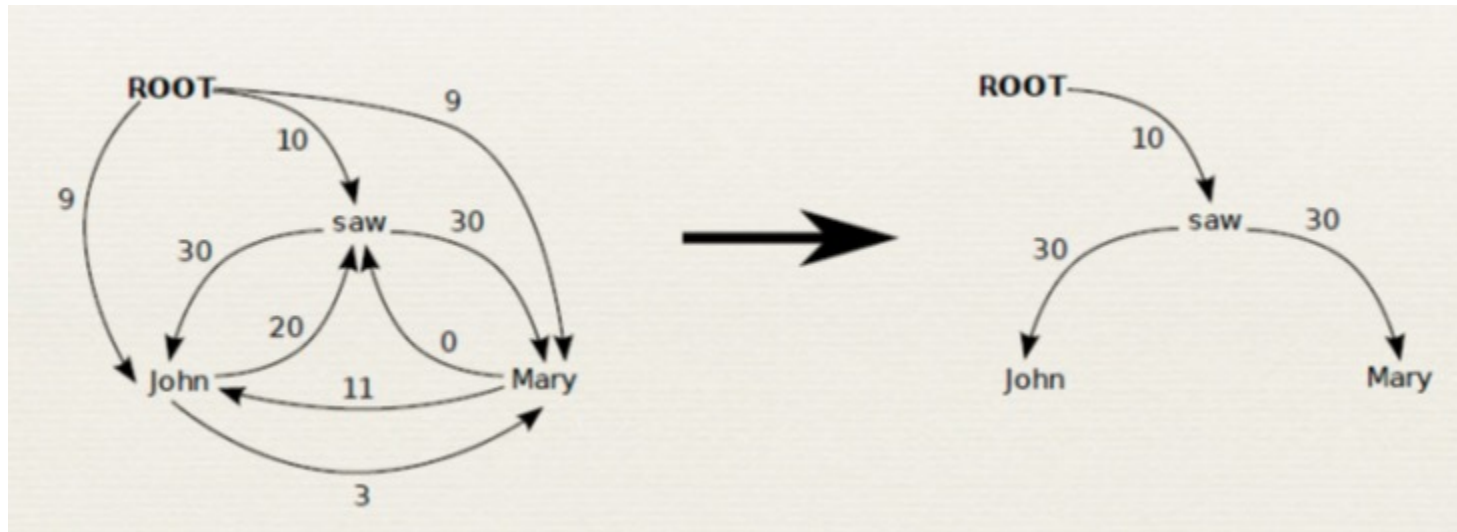


Graph-based Parsing

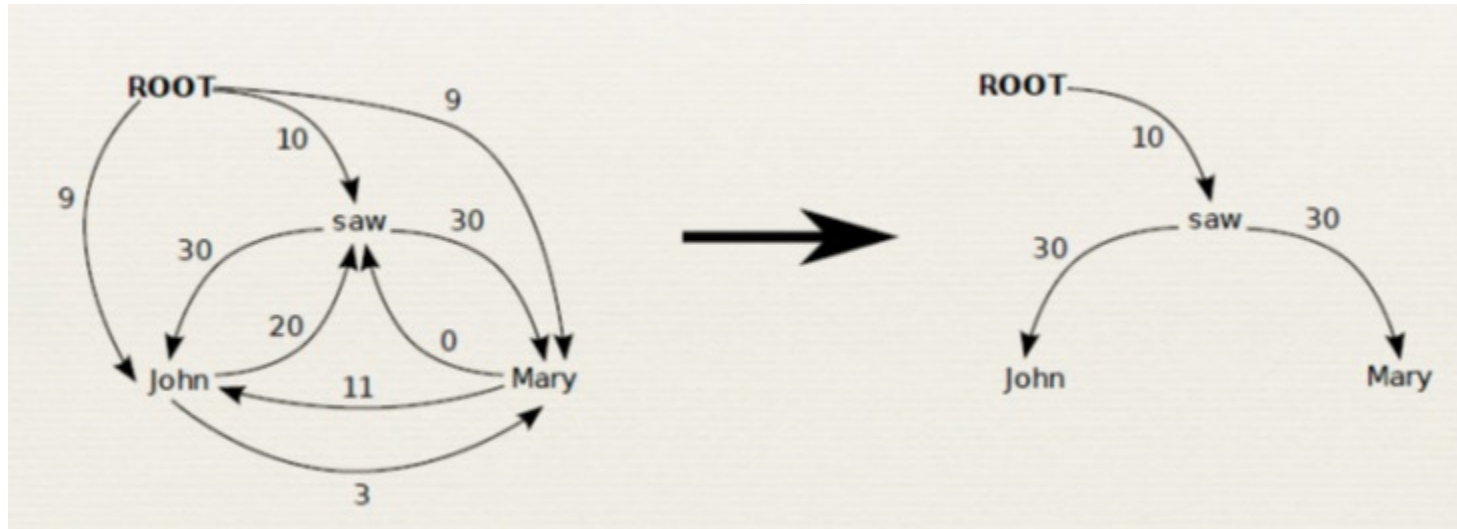
- Graph-based parsing addresses it as a structured prediction problem
- MST Parser:
 1. Score the arcs independently, based on how likely they are to appear in a parse
 2. Find the maximum directed spanning tree over the resulting weighted graph



Online Large-Margin Training of
Dependency Parsers
R. McDonald, K. Crammer, and
F. Pereira, *ACL 2005*

Graph-based Parsing

- It is also possible to have a multi-graph with several possible edges between each two nodes
 - Useful when using labeled dependency parsing – an edge is needed for each possible labels
 - During inference, it is possible to prune away all edges but the maximal, reducing the multi-graph to a graph



Online Large-Margin Training of
Dependency Parsers

R. McDonald, K. Crammer, and
F. Pereira, *ACL 2005*

MST Parser

Define a scoring function over all possible directed trees over $V = \{w_1, \dots, w_n, \textit{ROOT}\}$ where *ROOT* is the root of the tree. Let $\Phi : V^2 \times L \times S \rightarrow \{0, 1\}^d$, where L is the label set and S is the set of sentences (feature values can also be real numbers if needed), be a feature function over possible edges.

Let θ be the weight vector (the parameters of the model):

$$\textit{score}_\theta(v_1, v_2, l \mid x_{1:n}) = \theta^t \cdot \Phi(v_1, v_2, l, x_{1:n})$$

For a directed tree T define:

$$\textit{score}_\theta(T \mid x_{1:n}) = \sum_{(v_1, v_2, l) \in T} \textit{score}_\theta(v_1, v_2, l \mid x_{1:n})$$

MST Parser

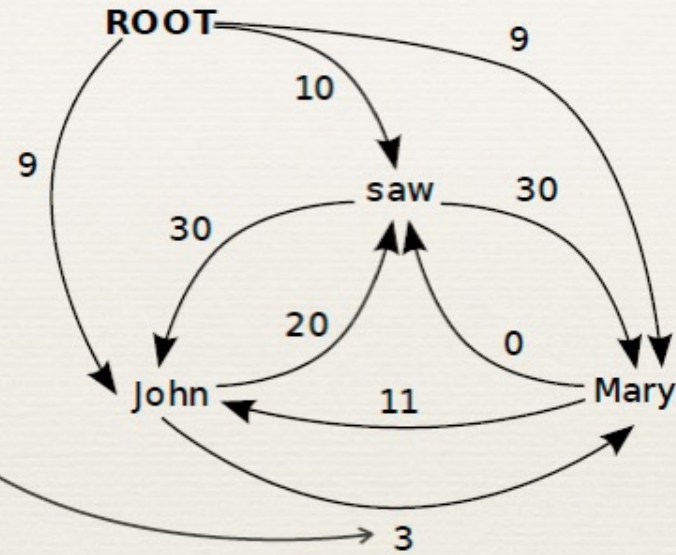
$\text{score}(\text{John} \rightarrow \text{Mary})$

$= w \cdot \mathbf{f}(\text{John} \rightarrow \text{Mary})$

Binary
Features

Head-POS=NOUN
Mod-POS=NOUN
Head-Word=John
Mod-Word=Mary
In-Between=VERB
Distance=2
Direction=Right
etc.

+conjunctions



MST Parser: Inference and Learning

- Note that inference is simply finding the maximum directed spanning tree
 - We can score each edge based on its features
 - This is done by the Chu-Liu Edmonds algorithm
- It is possible to define this model as log-linear:

$$Pr(T) = \frac{\exp(\sum_{(v_1, v_2, l) \in T} \theta^t \cdot \Phi(v_1, v_2, l))}{Z(V, \theta)}$$

- As usual, the gradient of the log-likelihood is given by:

$$\frac{\partial LL}{\partial \theta} = \sum_{i=1}^N \left[\sum_{(v_1, v_2, l) \in T_i} \Phi(v_1, v_2, l) - \mathbf{E}_T \left(\sum_{(v_1, v_2, l) \in T} \Phi(v_1, v_2, l) \right) \right]$$

MST Parser: Inference and Learning

$$\frac{\partial LL}{\partial \theta} = \sum_{i=1}^N \left[\sum_{(v_1, v_2, l) \in T_i} \Phi(v_1, v_2, l) - \mathbf{E}_T \left(\sum_{(v_1, v_2, l) \in T} \Phi(v_1, v_2, l) \right) \right]$$

- It is possible to compute the second term exactly, but the algorithm is not simple
- The Averaged Perceptron algorithm provides a simple and useful alternative, by replacing the expectation with a maximum

MST Parser: Inference and Learning

1. $\theta^{(0)} \leftarrow 0$
 2. **for** $r = 1 \dots N_{iterations}$
 3. **for** $i = 1 \dots N$
 4. $T' \leftarrow \operatorname{argmax}_T \sum_{(v_1, v_2, l) \in T} \operatorname{score}_\theta(v_1, v_2, l)$
 5. $\theta^{((r-1)N+i)} \leftarrow \theta^{((r-1)N+i-1)} + \eta \cdot \left(\sum_{(v_1, v_2, l) \in T_i} \Phi(v_1, v_2, l) - \sum_{(v_1, v_2, l) \in T'} \Phi(v_1, v_2, l) \right)$
 6. **return** $\frac{1}{N \cdot N_{iterations}} \sum_k \theta^{(k)}$
- T_i is the gold standard tree
- Learning Rate
- Feature function of T' instead of expectation over all possible trees